

Task 1 : MDP Planning

- **Input Handling :**

- The `parse_mdp()` function reads and parses the MDP file.
- All transitions are stored in a nested dictionary :

Python

```
transitions = {s:{}} for s in range(num_states)}
```

- **Howard's Policy Iteration (HPI) :**

- Policy Evaluation : uses iterative updates until converges
- Policy Improvement : Updates all states where a better action exists
- Evaluation Mode :
 - `evaluate_policyI` : full iterative evaluation until converges
 - `evaluate_policyF` : partial evaluation with fixed iterations (10) for speedup
- The process stops when the policy becomes stable or minimum iteration limit is reached
- Terminal states are excluded from updates by setting their policy to -1 and skipping evaluation.

- **Linear Programming (LP) :**

- Implemented using PuLP python lib with CBC solver
- Objective : $Minimize \sum V(s)$
- Constraints : $V(s) \geq R(s, a, s') + \gamma P(s, a, s')V(s')$ for all states **s** and actions **a**
- After solving the optimal policy is obtained by computing the greedy action for each state using $a^* = \arg \max_{a'} \sum P(s, a, s') [R(s, a, s') + \gamma V(s')]$

- **Command-Line Interface :**

None

```
--mdp <path>                #MDP file
--algorithm [hpi|lp]         #Default: hpi
--policy <path>              #Optional policy evaluation
```

- **Assumptions :**

- Terminal States are defined by "end", their value is always 0 and actions are -1
- Only non-zero transitions probabilities are provided
- Policy evaluation stops when the maximum value change $< 1e-9$
- If a state has no actions its policy is set to -1

- **Observations :**

- Algorithm Behaviour :
 - HPI :
 - Converges faster for smaller MDPs

- Computationally cheaper per iteration but may require several iterations
- LP :
 - Guarantees optimal solutions in one solver call
 - More memory intensive for large MDPs
 - Useful for deterministic or sparse transitions
- Performance :
 - For smaller MDPs both algorithms perform similarly
 - For larger MDPs HPI shows better performance and scalability due to reduced constraint overhead

Task 2 : MDP Modelling : Strategic Card Play

- MDP Formulation :
 - States :
 - A state is represented by the current set of cards in the agent's hand
 - The terminal state is represented as a unique key "TERMINAL" when the game ends either by **Stop** or by **Bust**
 - Only valid hands states (sum < Threshold) are included in the MDP
 - Implementation :
 - The encoder uses BFS from the empty hand to generate all possible hand states
 - deque is used to explore hand combinations efficiently while avoiding duplicates
 - Each hand is sorted and stored as a tuple to maintain consistent state representation
 - Actions :
 - Add(0) : Draw one card from the remaining deck
 - Swap(1-26) : Swap a specific card in hand with a card from the remaining deck
 - Stop(27) : End the game and collect the final score
 - Action Encoding :
 - Hearts numbered 1-13 -> actions 1-13
 - Diamonds numbered 1-13 -> actions 14-26
 - Implementation :
 - actionid4cards() maps a card to its corresponding action index
 - Swap actions are generated only for cards present in the hand
 - Transitions :
 - After an action, the next state is determined by the resulting hand
 - Probabilities are uniform across all remaining cards in the deck
 - Terminal states are reached either by Bust or Stop
 - Rewards :
 - Bust : 0

- Stop : sum of card values + bonus (is special sequence present)
 - Add/ Swap : 0 until terminal
- **Terminal States :**
 - Bust or Stop transitions always go to the “TERMINAL” state
 - Encoded as state 0 in the MDP
- **Strategy :**
 - BFs to enumerate all valid hands
 - Sorting hand tuples ensures deterministic state IDs making policy mapping reliable across testcases
- **Observations :**
 - Add is often preferred to increase hand value quickly
 - Swap actions are considered only when a low-value card can be replaced with higher card
 - LP v/s HPI :
 - LP :
 - Objective : *Minimize* $s \in S \sum V_s$
 - Constraints :

$$V_s \geq s' \sum P(s'|s, a)(R(s, a, s') + \gamma V_{s'}), \quad \forall s \in /terminal, \forall a \in A(s) \text{ and } V_s = 0, \forall s \in terminal$$
 - Since the final reward at the terminal is non-zero and the rest of the rewards are 0, and since the LP solver is trying to minimize the **sum(Vs)** it is choosing the smallest Vs satisfying the inequality.
 - So, $V_s \geq s' \sum p(s'|s, a)(\gamma V_{s'})$, this is always satisfied by $V_s = 0$ for all s, $0 \geq s' \sum p(s'|s, a)(\gamma \cdot 0) = 0$
 - Thus mathematically $V_s = 0$ for all s is a feasible solution and since the LP minimizes the sum it chooses all 0s
 - In LP, non-terminal states see only zero rewards for all actions, so the solver doesn't propagate the terminal “stop” rewards backward, leaving all V(s) at zero.
 - HPI :
 - It starts with an initial policy
 - It iteratively updates Vs using Bellman Equations, which eventually propagates the terminal rewards.
 - So for a terminal state T, $V(T) = R(T)$
 - So for its predecessor state s :

$$V(s) = s' \sum P(s'|s, a)(R(s, a, s') + \gamma V(s'))$$